

CÁC LƯU Ý QUAN TRỌNG VỀ Dictionary<TKey, TValue>

Mục tiêu tài liệu

Ôn lại các quy tắc cốt lõi khi dùng Dictionary trong C#: Key - Value, thêm, tìm, xóa, duyệt, nullable, encapsulation, exception và các lỗi thường gặp.

Chủ đề	Nội dung chính
Cấu trúc	Dictionary, Key, Value
Tra cứu	ContainsKey(), TryGetValue(), indexer
Quản lý	Add(), Remove(), Count, Clear()
Đóng gói	private readonly + các method có validation
Xử lý lỗi	ArgumentException, ArgumentNullException, InvalidOperationException

Tài liệu ôn tập - không kèm đáp án bài tập

MỤC LỤC

1. Dictionary lưu dữ liệu theo cặp Key - Value
2. Key phải duy nhất và nên ổn định
3. Add(), indexer và tính nhất quán của Key
4. ContainsKey() và TryGetValue()
5. Nullable khi tìm kiếm
6. Xóa bằng Remove(key)
7. Key/Value và Keys/Values
8. Encapsulation với Dictionary
9. private readonly có ý nghĩa gì?
10. Chuẩn hóa Key và phân biệt hoa thường
11. Chọn đúng loại exception
12. Các lỗi thường gặp
13. So sánh nhanh List và Dictionary
14. Checklist và bảng ghi nhớ

Quy tắc cốt lõi

Biết một khóa và cần tìm nhanh giá trị tương ứng là tình huống điển hình để dùng Dictionary.

1. Dictionary lưu dữ liệu theo cặp Key - Value

Dictionary không lưu dữ liệu như một danh sách đơn thuần. Mỗi phần tử gồm một khóa và một giá trị tương ứng.

```
Dictionary<string, Student> students =  
    new Dictionary<string, Student>();
```

Thành phần	Ý nghĩa
string	Kiểu dữ liệu của Key
Student	Kiểu dữ liệu của Value
"SV01"	Một Key cụ thể
student1	Value được liên kết với Key đó

```
students.Add("SV01", student1);  
students.Add("SV02", student2);
```

Hình dung

"SV01" -> Student An

"SV02" -> Student Bình

"SV03" -> Student Cường

Dictionary phù hợp khi hệ thống thường xuyên tra cứu bằng mã sinh viên, mã sản phẩm, username, mã nhân viên hoặc một khóa duy nhất khác.

2. Key phải duy nhất và nên ổn định

Trong một Dictionary, mỗi Key chỉ được xuất hiện một lần. Value có thể giống nhau nhưng Key không được trùng.

```
_students.Add("SV01", student1);  
_students.Add("SV01", student2); // lỗi do trùng Key
```

Trước khi thêm, nên kiểm tra Key:

```
if (_students.ContainsKey(student.Id))  
{  
    throw new InvalidOperationException(  
        $"Student with Id {student.Id} already exists.");  
}  
  
_students.Add(student.Id, student);
```

Key nên là dữ liệu duy nhất và ít thay đổi:

- Nên dùng: Student Id, Product Id, Employee Code, Username.
- Không nên dùng họ tên vì nhiều người có thể trùng tên.
- Không nên dùng dữ liệu thường xuyên thay đổi làm Key.

3. Key và thuộc tính Id phải thống nhất

Không nên tự viết một Key khác với Id của object:

```
_students.Add(  
    "SV99",  
    new Student("SV01", "An", 8.0));
```

Vấn đề

Dictionary Key = SV99 nhưng Student.Id = SV01. Dữ liệu bị mâu thuẫn và dễ tìm sai.

Nên dùng chính Id của object làm Key:

```
_students.Add(student.Id, student);
```

4. Add() và indexer khác nhau

Hai cú pháp đều có thể đưa dữ liệu vào Dictionary, nhưng hành vi khi Key đã tồn tại khác nhau.

Cú pháp	Key chưa có	Key đã có
dictionary.Add(key, value)	Thêm mới	Ném exception
dictionary[key] = value	Thêm mới	Ghi đè Value cũ

Ví dụ dùng Add():

```
_students.Add(student.Id, student);
```

Ví dụ dùng indexer:

```
_students[student.Id] = student;
```

Khuyến nghị khi mới học

Khi không muốn ghi đè nhầm, hãy kiểm tra ContainsKey() rồi dùng Add().

Indexer cũng dùng để lấy Value:

```
Student student = _students["SV01"];
```

Nhưng nếu Key không tồn tại, chương trình sẽ phát sinh KeyNotFoundException. Vì vậy không nên dùng indexer khi chưa chắc chắn Key tồn tại.

5. ContainsKey() và TryGetValue()

ContainsKey() chỉ kiểm tra xem Key có tồn tại hay không.

```
if (_students.ContainsKey(id))
{
    Console.WriteLine("Id already exists.");
}
```

TryGetValue() vừa kiểm tra Key, vừa lấy Value trong cùng một thao tác.

```
if (_students.TryGetValue(
    id,
    out Student? student))
{
    student.DisplayInfo();
}
else
{
    Console.WriteLine("Student not found.");
}
```

Nhu cầu	Nên dùng
Chỉ cần biết Key có tồn tại	ContainsKey(key)
Cần lấy Value tương ứng	TryGetValue(key, out value)

Ý nghĩa của out:

TryGetValue() đưa Value tìm được ra biến được khai báo sau từ khóa out, đồng thời trả về true hoặc false.

```
bool found = _students.TryGetValue(
    "SV02",
    out Student? student);
```

Kết quả

Tìm thấy: found = true và student chứa object.

Không tìm thấy: found = false và student là null.

6. Kết quả tìm kiếm có thể là null

Một method tìm kiếm thường trả về kiểu nullable vì không phải lúc nào cũng tìm thấy dữ liệu.

```
public Student? FindStudentById(string id)
{
    if (string.IsNullOrEmpty(id))
    {
        throw new ArgumentException(
            "Id cannot be null or whitespace.",
            nameof(id));
    }

    string normalizedId = id.Trim();

    if (!_students.TryGetValue(
        normalizedId,
        out Student? student))
    {
        return student;
    }

    return null;
}
```

Biến nhận kết quả cũng nên có dấu ?:

```
Student? foundStudent =
    directory.FindStudentById("SV02");
```

Sau đó phải kiểm tra null trước khi sử dụng object.

7. Xóa trực tiếp bằng Remove(key)

```
public bool RemoveStudentById(string id)
{
    if (string.IsNullOrEmpty(id))
    {
        throw new ArgumentException(
            "Id cannot be null or whitespace.",
            nameof(id));
    }

    return _students.Remove(id.Trim());
}
```

Kết quả Remove()	Ý nghĩa
true	Key tồn tại và phần tử đã được xóa
false	Key không tồn tại

Không cần làm

Không cần lấy `_students[id]` trước khi xóa. Nếu Key không tồn tại, cách đó sẽ gây `KeyNotFoundException`.

8. Key/Value và Keys/Values

Khi duyệt toàn bộ Dictionary, mỗi phần tử là một KeyValuePair.

```
foreach (KeyValuePair<string, Student> item
        in _students)
{
    Console.WriteLine(item.Key);
    item.Value.DisplayInfo();
}
```

Cú pháp	Ý nghĩa
item.Key	Một Key của cặp hiện tại
item.Value	Một Value của cặp hiện tại
dictionary.Keys	Collection chứa tất cả Key
dictionary.Values	Collection chứa tất cả Value

Duyệt riêng tất cả Key:

```
foreach (string key in _students.Keys)
{
    Console.WriteLine(key);
}
```

Duyệt riêng tất cả Value:

```
foreach (Student student in _students.Values)
{
    student.DisplayInfo();
}
```

Ghi nhớ

Key và Value thuộc về một cặp hiện tại. Keys và Values thuộc về toàn bộ Dictionary.

9. Encapsulation với Dictionary

Không nên công khai trực tiếp Dictionary nội bộ vì code bên ngoài có thể bỏ qua validation, xóa toàn bộ hoặc ghi đè dữ liệu.

Không nên:

```
public Dictionary<string, Student> Students { get; }
```

Bên ngoài có thể làm:

```
directory.Students.Clear();  
directory.Students["SV01"] = anotherStudent;  
directory.Students.Add("ABC", student);
```

Nên đóng gói:

```
private readonly Dictionary<string, Student>  
    _students;
```

Và cung cấp các method có kiểm soát:

- AddStudent(Student student)
- FindStudentById(string id)
- RemoveStudentById(string id)
- DisplayStudents()

Mục đích của encapsulation

Mọi thay đổi phải đi qua method của class để kiểm tra null, Key trùng, dữ liệu rỗng và các quy tắc nghiệp vụ khác.

Bảo vệ cả object bên trong:

Dù collection được bảo vệ, object Student vẫn có thể bị sửa sai nếu property dùng public set. Vì vậy nên dùng get-only hoặc private set và cung cấp method cập nhật có validation.

10. Ý nghĩa của private readonly

Từ khóa	Tác dụng
private	Code bên ngoài không truy cập trực tiếp field
readonly	Field không được gán sang Dictionary khác sau khi khởi tạo

readonly không khóa nội dung bên trong Dictionary. Class vẫn có thể thêm hoặc xóa phần tử:

```
_students.Add(id, student);  
_students.Remove(id);  
_students.Clear();
```

Cách nhớ

readonly bảo vệ reference của collection, không làm collection trở thành bất biến.

11. Chuẩn hóa Key trước khi tìm hoặc xóa

```
string normalizedId = id.Trim();
```

Sau đó dùng normalizedId để tìm hoặc xóa:

```
_students.TryGetValue(  
    normalizedId,  
    out Student? student);
```

```
_students.Remove(normalizedId);
```

Nếu không Trim(), "SV01" và " SV01 " là hai chuỗi khác nhau.

Không phân biệt hoa thường:

```
_students = new Dictionary<string, Student>(  
    StringComparer.OrdinalIgnoreCase);
```

Khi đó SV01, sv01 và Sv01 được xem là cùng một Key. Đây là lựa chọn hữu ích với mã hoặc username tùy quy tắc hệ thống.

12. Chọn đúng loại exception

Tình huống	Exception phù hợp
Object student được truyền vào là null	ArgumentNullException
Id là null, rỗng hoặc whitespace	ArgumentException
GPA nằm ngoài 0 đến 10	ArgumentOutOfRangeException
Id đã tồn tại trong Dictionary	InvalidOperationException

Object null:

```
throw new ArgumentNullException(  
    nameof(student),  
    "Student cannot be null.");
```

Chuỗi không hợp lệ:

```
throw new ArgumentException(  
    "Id cannot be null or whitespace.",  
    nameof(id));
```

Ngoài phạm vi:

```
throw new ArgumentOutOfRangeException(  
    nameof(gpa),  
    gpa,  
    "GPA must be between 0 and 10.");
```

Trùng Key:

```
throw new InvalidOperationException(  
    $"Student with Id {student.Id} already exists.");
```

nameof()

Trong exception kiểm tra tham số, nameof(...) nên trỏ đến parameter của method hoặc constructor, ví dụ nameof(id) hoặc nameof(student).

13. Các lỗi thường gặp

Lấy bằng indexer khi Key chưa chắc tồn tại

`_students[id]` có thể ném `KeyNotFoundException`. Dùng `TryGetValue()`.

Dùng Key khác với `object.Id`

Dữ liệu bị mâu thuẫn. Dùng `_students.Add(student.Id, student)`.

Bắt sai exception khi Key trùng

`AddStudent` ném `InvalidOperationException` thì `catch` cũng phải bắt loại phù hợp.

Biến nhận kết quả không nullable

Method trả `Student?` thì biến nhận cũng nên là `Student?`.

Không `Trim()` Key

Khoảng trắng thừa làm việc tìm và xóa thất bại.

Để Dictionary là public

Code ngoài có thể `Clear()`, `Add()` hoặc ghi đè không qua validation.

Không kiểm tra Dictionary rỗng

Display method nên xử lý `Count == 0` và thông báo rõ ràng.

Chỉ in `item.Value` khi đề yêu cầu cả Key

Duyệt `KeyValuePair` thì có thể hiển thị `item.Key` và `item.Value`.

14. So sánh nhanh List và Dictionary

Nội dung	List	Dictionary
Cấu trúc	Danh sách phần tử	Cặp Key - Value
Tìm theo Id	Thường phải foreach	TryGetValue(key, out value)
Kiểm tra trùng Id	Duyệt và so sánh	ContainsKey(key)
Xóa theo Id	Tìm object rồi Remove(object)	Remove(key)
Quy tắc trùng	Có thể chứa phần tử trùng	Key bắt buộc duy nhất
Mục tiêu	Duyệt danh sách, giữ thứ tự	Tra cứu theo khóa

Chọn Dictionary khi

Bạn biết một Key cụ thể và thường xuyên cần tìm, cập nhật hoặc xóa Value tương ứng theo Key đó.

Dictionary không thay thế database:

Dictionary lưu dữ liệu trong RAM và dữ liệu thường mất khi ứng dụng dừng. Trong .NET, nó thường dùng cho dữ liệu tạm, cache, mapping, thống kê hoặc xử lý dữ liệu sau khi đọc từ database.

15. Checklist trước khi nộp bài Dictionary

- Dictionary được khai báo private readonly.
- Constructor khởi tạo Dictionary rỗng.
- Object truyền vào Add không được null.
- Dùng object.Id làm Key.
- Dùng ContainsKey() để kiểm tra Key trùng.
- Key trùng ném InvalidOperationException.
- Method tìm kiếm validation Id và dùng TryGetValue().
- Method tìm kiếm trả về kiểu nullable, ví dụ Student?.
- Biến nhận kết quả tìm kiếm cũng là Student?.
- Method xóa dùng Remove(key) và trả về bool.
- Không dùng dictionary[id] khi chưa chắc Key tồn tại.
- Duyệt bằng KeyValuePair nếu cần hiển thị cả Key và Value.
- Chuỗi được Trim() trước khi lưu, tìm hoặc xóa.
- Property của object không dùng public set tùy tiện.
- Program có try-catch đúng loại exception khi kiểm thử dữ liệu sai.

Bảng ghi nhớ nhanh

Nhu cầu	Cú pháp
Thêm	Add(key, value)
Kiểm tra Key	ContainsKey(key)
Tìm và lấy Value	TryGetValue(key, out value)
Xóa	Remove(key)
Số phần tử	Count
Tất cả Key	Keys
Tất cả Value	Values
Xóa toàn bộ	Clear()
Duyệt từng cặp	KeyValuePair

7 quy tắc cốt lõi

1. Key phải duy nhất.
2. Không dùng `dictionary[key]` khi chưa chắc Key tồn tại.
3. Cần lấy Value thì ưu tiên `TryGetValue()`.
4. Xóa trực tiếp bằng `Remove(key)`.
5. Giữ Dictionary là private readonly.
6. Mọi thay đổi đi qua method có validation.
7. Key và thuộc tính Id của object phải thống nhất.